

POSTMAN COLLECTION TO PYTEST MIGRATOR

Project Documentation

Submitted By

Team Members

1. **Mohamed Ashiq T**
B.E. Computer Science and Engineering (CSE)
 2. **Harini R**
B.E. Electronics and Communication Engineering (ECE)
 3. **Dhiviya K**
B.E. Electronics and Communication Engineering (ECE)
 4. **Nithish S**
B.E. Computer Science and Engineering (CSE)
-

1. Abstract

Postman Collection to Pytest Migrator is an AI-assisted automation platform designed to simplify the migration of API test cases from Postman to Pytest. Organizations commonly use Postman for API testing and Pytest for automation testing. However, converting Postman collections into Pytest scripts is often a manual, repetitive, and error-prone task.

The proposed system automates this migration process by parsing Postman Collection JSON files, extracting API requests and assertions, validating collection structures, and generating executable Pytest scripts. The system also provides recommendations, test cases, and downloadable reports to improve testing quality.

A Hybrid Conversion Engine is implemented, combining Rule-Based Conversion and AI-Assisted Conversion using Google Gemini API. This approach ensures efficient handling of both simple and complex assertion logic.

The solution improves productivity, reduces migration effort, and accelerates automation adoption for QA teams and software organizations.

2. Introduction

API testing is a critical phase in software development. Many organizations use Postman for API validation and testing.

When teams migrate towards automation frameworks such as Pytest, developers and QA engineers must manually rewrite Postman assertions into Python code.

This creates challenges such as:

- Increased development time
- Human errors

- Maintenance difficulties
- Repetitive work

The Postman Collection to Pytest Migrator addresses these challenges by providing an automated migration platform.

3. Problem Statement

QA engineers frequently create API tests inside Postman collections.

Challenges include:

- Manual conversion of Postman tests to Pytest
- Time-consuming migration process
- High possibility of errors
- Lack of automated recommendation generation
- Difficult maintenance of large collections

There is a need for a system that automatically converts Postman collections into executable Pytest scripts while generating useful testing recommendations.

4. Objectives

The primary objectives are:

1. Upload Postman Collection JSON files.
 2. Parse APIs and assertions automatically.
 3. Validate collection structure.
 4. Convert assertions into Pytest format.
 5. Generate executable Python test scripts.
 6. Generate recommendations and test cases.
 7. Provide downloadable outputs.
 8. Ensure secure multi-user access.
-

5. Proposed Solution

The system automates the migration process through:

- Collection Upload
- Collection Parsing
- Validation

- Hybrid Conversion Engine
- Recommendation Generation
- Test Case Generation
- Pytest Script Generation
- Download Center

The generated scripts can be directly used within automation projects.

6. System Architecture

User

↓

Authentication Module

↓

Dashboard

↓

Collection Upload

↓

Parser Engine

↓

Validation Engine

↓

Hybrid Conversion Engine

↓

Recommendation Engine

↓

Pytest Generator

↓

Report Generator

↓

Download Center

7. Technology Stack

Frontend

- HTML5
- CSS3
- JavaScript

Backend

- Python
- Flask

Database

- SQLite

AI Integration

- Google Gemini API

Testing

- Pytest

Deployment

- Render Cloud Platform
-

8. Modules Description

Authentication Module

Functions:

- Registration
- Login
- Logout
- Session Management

Security Features:

- Password Hashing
 - User Authentication
 - Protected Routes
-

Collection Upload Module

Functions:

- Upload Postman Collection
 - Validate JSON Format
 - Store Collection Information
-

Parser Module

Extracts:

- API Endpoints
- HTTP Methods

- Headers
 - Request Body
 - Query Parameters
 - Assertions
-

Validator Module

Performs:

- JSON Validation
 - Missing Variable Detection
 - Structure Verification
 - Collection Integrity Checks
-

Hybrid Conversion Engine

Rule-Based Engine

Converts standard assertions:

Example:

Postman:

```
pm.response.to.have.status(200)
```

Pytest:

```
assert response.status_code == 200
```

AI-Assisted Engine

Uses Google Gemini API to convert:

- Complex JavaScript Logic
 - Conditional Assertions
 - Advanced Validation Rules
-

Recommendation Engine

Generates:

- Positive Tests
- Negative Tests
- Boundary Tests

- Security Recommendations

Report Generation Module

Provides:

- Conversion Summary
- Generated Scripts
- Recommendations
- Test Cases

9. Database Design

Tables Used:

Users

- id
- full_name
- email
- password_hash
- created_at

Uploaded Collections

- id
- user_id
- collection_name
- uploaded_at

API Details

- id
- collection_id
- api_name
- method
- endpoint

Generated Scripts

- id
- user_id

- script_content

Generated Test Cases

- id
- user_id
- testcase_name

AI Recommendations

- id
- user_id
- recommendation

10. Security Features

Implemented Security Measures:

- Password Hashing using Werkzeug
- Session-Based Authentication
- User Data Isolation
- Protected Routes
- Input Validation

Each user's data is isolated using user_id filtering.

11. Sample Workflow

1. User Registration
2. User Login
3. Upload Collection
4. Parse Collection
5. Validate Collection
6. Generate Pytest Script
7. Generate Recommendations
8. View Reports
9. Download Output Files

12. Testing

The system was tested for:

- Registration
- Login
- Collection Upload
- Collection Validation
- Parser Accuracy
- Assertion Extraction
- Pytest Generation
- Download Functionality
- Theme Switching
- User Data Isolation

Result:

All critical functionalities executed successfully.

13. Deployment

The application is deployed using Render.

Deployment URL:

<https://postman-pytest-migrator-2.onrender.com>

14. Advantages

- Reduces manual effort
 - Faster migration
 - Improved accuracy
 - AI-assisted conversion
 - Secure user management
 - Downloadable automation scripts
-

15. Limitations

- Complex custom JavaScript may require AI assistance.
- SQLite is suitable only for small to medium workloads.
- Generated scripts may require manual review in advanced scenarios.

16. Future Scope

- PostgreSQL Integration
- Batch Collection Processing
- CI/CD Integration
- AI Optimization Suggestions
- Enterprise Scalability

17. Conclusion

The Postman Collection to Pytest Migrator successfully automates the migration of Postman API test collections into Pytest automation scripts. By combining Rule-Based Conversion and AI-Assisted Conversion, the system reduces manual effort, improves testing productivity, and accelerates automation adoption. The platform provides a complete workflow from collection upload to downloadable automation scripts, making it a practical solution for modern QA teams.